

Kiste v0.9.3 Version Contract Spec

Full CLI/API/YAML contract to prevent version drift

Status	Draft canonical version contract
Repository source of truth	KisteBox/kiste_spec
Target implementation repo	KisteBox/kiste-py
Release	0.9.3
Lifecycle	Read -> Inspect -> Plan -> Review -> Deploy -> Monitor

1. Core Rule

Every Kiste version must ship with a full version contract. A release is not complete if it only describes features.

A release is complete only when CLI, Python API, YAML/config, schema IDs, output layout, data models, doctor groups, adapter behavior, deployment safety, monitoring, migration, and acceptance tests agree.

Kiste must not allow the spec to say one thing while the implementation exposes another.

2. Anti-Drift Principle

Kiste must treat every release as a contract, not a note.

Bad: v0.9.3 adds CNCF and PyTorch integration.

Good: v0.9.3 defines the exact CLI, API, YAML, schemas, outputs, doctor checks, adapter rules, and acceptance tests for CNCF and PyTorch integration.

3. Required Sections for Every Version Spec

Each version spec must contain: release identity, release theme, design principles, non-goals, lifecycle contract, CLI contract, Python API contract, YAML/config contract, schema ID contract, output/state layout contract, data model contract, subsystem CLI contract, adapter/plugin contract, doctor/inspection contract, deployment safety contract, monitor contract, compatibility rules, migration rules, implementation checklist, and acceptance tests.

4. Release Identity

Every version must declare: release version, status, source-of-truth repository, target implementation repository, schema family, compatibility policy, and deprecation policy.

Release identity example: version 0.9.3, status canonical, source_of_truth KisteBox/kiste_spec, target_implementation KisteBox/kiste-py, schema_family kiste.v0.9.3.

5. Lifecycle Contract

Every version must say how it supports the Kiste lifecycle: Read -> Inspect -> Plan -> Review -> Deploy -> Monitor.

Each stage must define inputs, outputs, commands, API functions, state files, and safety rules.

For v0.9.3, the canonical lifecycle commands are: kiste read, kiste inspect, kiste plan, kiste review, kiste deploy, and kiste monitor.

Existing commands may remain as aliases or subsystem commands: kiste scan as read alias; kiste doctor as inspect/monitor engine; kiste workspace as workspace subsystem; kiste argocd as GitOps subsystem.

6. CLI Contract

Every version must define the exact public CLI.

Canonical v0.9.3 lifecycle CLI: kiste read; kiste inspect; kiste plan; kiste review; kiste deploy; kiste monitor.

Subsystem CLI: kiste workspace, repo, catalog, adapter, foundation, data, blob, image, secrets, target, gitops, argocd, cloud, policy, cost, module, config, release, doctor.

Lifecycle CLI is the product journey. Subsystem CLI is the expert control surface.

7. Python API Contract

Every version must define its public Python API.

Required public functions for v0.9.3: kiste.read(...), kiste.inspect(...), kiste.plan(...), kiste.review(...), kiste.deploy(...), and kiste.monitor(...).

Backward-compatible aliases may exist: kiste.scan(...) as alias for read and kiste.run_doctor(...) as engine for inspect/monitor.

8. YAML Contract

Every version must define valid YAML fields and allowed top-level keys.

For v0.9.3, the YAML must support workspace, lifecycle, foundations, delivery, adapters, security, observability, catalog, deploy, and monitor sections.

The lifecycle section must support require_review_before_deploy, require_approved_plan, default_scope, and default_profile.

The foundations section must support CNCF and PyTorch policies with fit-for-purpose, secure-by-default, and cost-effective gates.

9. Schema ID Contract

Every version must define schema IDs and avoid accidental old schema strings.

v0.9.3 schema IDs: kiste.config/v0.9.3, kiste.workspace/v0.9.3, kiste.read/v0.9.3, kiste.inspect/v0.9.3, kiste.plan/v0.9.3, kiste.review/v0.9.3, kiste.deploy/v0.9.3, kiste.monitor/v0.9.3, kiste.adapter/v0.9.3, and kiste.foundation/v0.9.3.

Old schema IDs are allowed only when explicitly marked as legacy compatibility.

10. Output Layout Contract

Every version must define the .kiste/ state layout.

v0.9.3 output roots: .kiste/read/, .kiste/inspect/, .kiste/doctor/, .kiste/plans/, .kiste/reviews/, .kiste/deploy/, .kiste/monitor/, .kiste/workspace/, and .kiste/exports/.

Read writes facts. Inspect writes reports. Plan writes proposed changes. Review writes decisions. Deploy writes execution reports. Monitor writes continuous status and events.

11. Data Model Contract

Every version must define stable report models: ReadReport, InspectionReport, PlanReport, ReviewReport, DeploymentReport, MonitorReport, DoctorReport, AdapterReport, and FoundationDecisionReport.

Each report must include schema, version, status, generated_at, checks, findings, and next_actions.

12. Foundation Integration Contract

For v0.9.3, Kiste integrates CNCF and PyTorch Foundation tools with opinion.

Core principle: CNCF and PyTorch tools must be fit for purpose, secure by default, and cost effective.

Kiste must not recommend tools only because they are popular.

Decision levels: adopt, defer, avoid, manual.

13. Adapter Contract

Every adapter must declare name, ecosystem, capabilities, load policy, dependency policy, state output, and supported lifecycle stages.

Adapter lifecycle: detect -> load -> run -> idle -> unload/cache.

Kiste must use detect-before-load, lazy loading by default, worker isolation for heavy adapters, and no execution before approved plan.

Heavy adapters include PyTorch, Pulumi, Ansible, large cloud SDK adapters, and custom execution adapters.

14. Doctor Contract

Every version must define doctor groups.

v0.9.3 doctor groups include config, repo, workspace, catalog, adapter, foundation, cncf, pytorch, target, gitops, argocd, cloud, data, blob, image, secrets, policy, cost, security, performance, drift, deploy, monitor, and release.

kiste doctor remains the validation engine. kiste inspect uses doctor before deployment. kiste monitor uses doctor continuously after deployment.

15. Deployment Safety Contract

Deploy must never bypass Review.

kiste deploy must require an approved plan. Allowed: kiste deploy --approved-plan .kiste/reviews/latest.approved.json.

Avoid bare kiste deploy. Subsystems must not bypass lifecycle safety. For example, kiste argocd sync should either not exist or require an approved plan.

16. Monitor Contract

Monitor is continuous validation after deployment.

It must detect health failure, security regression, policy violation, cost anomaly, performance regression, configuration drift, runtime drift, and data/model drift.

Monitor must be able to trigger a new cycle: Monitor -> Inspect -> Plan -> Review -> Deploy -> Monitor.

Required monitor commands: kiste monitor --once, kiste monitor --watch, kiste monitor --interval 60, and kiste monitor --scope workspace.

17. Compatibility Rules

Every version must state what is new, what is deprecated, what remains as alias, what breaks, and what migrates automatically.

For v0.9.3: kiste scan remains as read alias; kiste doctor remains as validation engine; kiste workspace remains as subsystem command; kiste argocd remains a first-class subsystem command; top-level lifecycle becomes canonical.

18. Migration Rules

Every version must provide migration behavior.

For v0.9.3, kiste config migrate must migrate config to kiste.config/v0.9.3.

Migration must update version fields, schema IDs, allowed YAML keys, output layout expectations, doctor groups, and deprecated command warnings.

19. Acceptance Tests

Acceptance tests for v0.9.3: kiste --version returns 0.9.3; kiste config schema returns kiste.config/v0.9.3; kiste read writes .kiste/read/; kiste inspect writes .kiste/inspect/; kiste plan writes .kiste/plans/latest.json; kiste review can approve/reject/modify a plan; kiste deploy refuses to run without approved plan; kiste monitor --once writes .kiste/monitor/status.json; kiste doctor supports cncf and pytorch groups; kiste adapter detect does not load heavy SDKs unnecessarily; kiste argocd remains a subsystem command; kiste scan remains compatible as read alias.

20. Final Rule

A Kiste release is complete only when: Spec version = package version = CLI version = schema version = output version = doctor version.

If any of these disagree, the release has drift.

Kiste must doctor itself.

Appendix A - Canonical CLI Examples

```
kiste read --scope workspace
kiste inspect --scope workspace --profile strict
kiste plan --target kubernetes --mode gitops
kiste review .kiste/plans/latest.json
kiste deploy --approved-plan .kiste/reviews/latest.approved.json
kiste monitor --scope workspace --watch
```

```
# Expert subsystem examples
kiste workspace graph
kiste catalog export --format backstage
kiste foundation recommend
kiste adapter detect
kiste data lineage
kiste blob inspect s3://bucket/model.pt
kiste gitops validate
kiste argocd app list
kiste cloud cost estimate
kiste secrets scan
```

Appendix B - Python API Example

```
import kiste

read_report = kiste.read(path='.')
inspection = kiste.inspect(read_report, scope='workspace', profile='strict')
plan = kiste.plan(inspection, target='kubernetes', mode='gitops')
review = kiste.review(plan, decision='approve')
deployment = kiste.deploy(review)
monitor = kiste.monitor(scope='workspace', once=True)
```

Appendix C - YAML Contract Example

```
kiste: "0.9.3"
schema: "kiste.workspace/v0.9.3"
kind: Workspace

workspace:
  name: ai-product-platform
  owner: group:default/platform-team
  lifecycle: experimental

lifecycle:
  mode: read-inspect-plan-review-deploy-monitor
  require_review_before_deploy: true
  require_approved_plan: true
  default_scope: workspace

foundations:
  cncf:
    enabled: true
```

policy: minimal-secure-stack
gates:
 fit_for_purpose: required
 secure_by_default: required
 cost_effective: required
pytorch:
 enabled: true
 policy: workload-fit
 gates:
 runtime_fit: required
 hardware_fit: required
 cost_effective: required

delivery:
 target: kubernetes
 mode: gitops
 package: kustomize
 cd: argocd

adapters:
 loading_policy: lazy
 detect_before_load: true
 isolate_heavy_adapters: true
 unload_after_seconds: 300

monitor:
 enabled: true
 mode: continuous
 interval_seconds: 60